

PipelineInsights — Review of Checkpoints 4, 5, and 6

Reviewed: September 16, 2026

New code: etl/tennis/sackmann.py (663 lines), features/build_features.py (997), features/transforms.py (310), migrations 009–010, four new test files (14 tests, all passing)

Verdict: The feature engineering is correct. I verified the leakage boundaries independently rather than trusting the tests, and they hold under every test I could design. This is the best work the agent has produced.

There are two code bugs, one config inconsistency, and one problem that has nothing to do with the code: **your data source is four months stale and the repository my earlier reviews told you to use no longer exists**. That last one is the most important thing in this document.

Part 1 — The leakage audit is real

This was the checkpoint where a project like this usually breaks. It didn't.

features/transforms.py centers on one function, `_strict_prior_aggregate`, that every feature primitive routes through. It walks each player's history in date order, and for each event date it computes the aggregate from a history list containing only strictly earlier dates, then appends the current date's rows *after* assignment. It carries two assertions inside the loop, with this comment:

```
# Including today's result is the most common beginner mistake in
# sports modeling and produces spectacular fake accuracy.
assert all(item.row_index != row_index for item in selected)
assert all(item.event_date < event_date for item in selected)
```

I did not take that on faith. I built small frames where I know the answer by hand:

Test	Expected	Result
Player with aces 10, 20, 30, 40 on four dates; rolling_mean_3	[NaN, 10, 15, 20] — match 4 sees 10/20/30, never 40	✓ exact
Two matches forced onto the same date	Both see only the earlier match, neither sees the other	✓
Inflate a row's <i>own</i> aces to 999	That row's own feature must not move from 20.0	✓ unchanged
opponent_rolling — A vs B twice	In match 2, A's opponent-form feature = B's <i>prior</i> aces (7), not B's aces in this match (70)	✓ 7.0

The perturbation test is the one that matters. If a feature changes when you alter the outcome it is supposed to predict, the feature contains that outcome. It doesn't.

Three more things it got right that are easy to get wrong:

Same-date ties are excluded in both directions. Two matches sharing an `event_date` cannot see each other. Slightly conservative — a Wednesday match genuinely played after a Tuesday match with the same estimated date loses that history — but conservative is the correct direction to err.

History is filtered by `match_completed`. Retirements and walkovers are excluded from rolling windows via `eligibility_col`, rather than being included with missing stats silently coerced to zero. A retirement at 2-1 in the first set would otherwise drag a player's ace average down for the next ten matches.

Missing values stay missing. `_safe_ratio` divides only where the denominator is positive and replaces infinities with NaN. `rolling_std` returns NaN rather than 0 when fewer than two observations exist. Nothing in the ETL converts an absent stat into a zero, which is the single most common way tennis feature tables get quietly poisoned.

The architecture is right

`config/tennis.yaml` now defines features as data — rolling windows, aggregations, category splits, rate and percentage definitions, head-to-head, opponent form, career-to-date, context columns, and derived columns are all declared in YAML and read by a generic builder. There is no `if sport == "tennis"` anywhere in `build_features.py`.

That is the generalization principle from your project instructions, implemented at Checkpoint 6 rather than deferred to Week 25. When you reach MLB, the sport-specific work should genuinely be a new ETL module and a new YAML file.

The config also carries a `descriptions:` block mapping every feature to a plain-English explanation, and `print_report` uses it to print one real example row — "player X before match Y on date Z" with each feature value and its meaning. For someone learning this material while building it, that is worth more than the code.

Migration 010 adds `has_sufficient_history` and `is_training_eligible` flags, so rows with thin history are marked rather than silently trained on. `minimum_history: 5` in the config drives it.

Part 2 — ❌ The data problem

This is not a code defect and the agent handled it defensibly, but it changes your plan.

The canonical repository is gone. `github.com/JeffSackmann/tennis_atp` returns 404. Not rate-limited, not moved to a main branch — the GitHub API returns `"message": "Not Found"` for the repo itself. Every earlier review I gave you named that repo as your source. That advice is now wrong, and I should have checked before repeating it.

What the code uses instead: `Kadantte/tennis_atp`, which is a fork of `jasonmauss/tennis_atp`, which is presumably a fork of the original. A fork of a fork, last pushed June 8, 2026.

How stale that is. I pulled the files:

File	Rows	Coverage
atp_matches_2025.csv	2,944	complete, through Next Gen Finals (Dec 2025)
atp_matches_2026.csv	1,449	Jan 4 → May 25, 2026 (Roland Garros)

Today is September 16, 2026. **Your data stops at the French Open.** Missing: the entire grass season including Wimbledon, the North American hardcourt swing, and the US Open. Roughly four months, and it is the four months most relevant to any player's current form.

Ace rates are populated on 82% of 2026 rows, which is normal for Sackmann-lineage data — Challengers and some ATP 250s lack serve stats. Your `match_completed` and null-handling logic deals with that correctly.

Why this matters more than it looks

Your Checkpoint 5 design captures odds snapshots nightly for matches that have not been played yet. Your feature builder computes a player's form from completed matches. If completed-match data stops in May, then starting today every prediction you generate uses form features that are four months out of date — while the odds you compare them against are current. You would be measuring your model against a market that knows about Wimbledon and the US Open when your model does not.

That will not show up as an error. It will show up as a model that looks mediocre for reasons you cannot diagnose.

What to do

Do not paper over this. Before writing another line of model code, settle the data source, because everything downstream inherits it.

1. **Find out what happened to the original.** Sackmann's data has been the backbone of public tennis analytics for a decade; a repo that size disappearing is unusual and there is likely a successor, a new org, or an announcement. Worth twenty minutes of searching before you commit to a mirror.
2. **Survey the live forks.** The fork network will have members that are still tracking current results. You want the most recently pushed one with 2026 data through last week, and you want to check its `pushed_at` before trusting it.
3. **Pin whatever you choose, and monitor it.** Your ETL already records `source` in `SeasonStats`. Add a freshness assertion: if the newest `tourney_date` in the current season's file is more than 21 days old, fail the ingestion run loudly instead of loading stale data quietly. A pipeline that silently ingests old data is worse than one that stops.
4. **Consider decoupling.** Long term, relying on a volunteer-maintained mirror for a project that has to still be running in September 2027 is a real risk. Since you already capture odds snapshots nightly, consider also capturing completed-match results nightly from a second source, so a mirror going dark costs you a gap rather than your history.

One thing in the code's favor: `etl/tennis/sackmann.py` centralizes the source in two constants (`SOURCE_REPOSITORY` and the base URL) and caches downloads under `.cache/`. Switching sources is a two-line change plus a column mapping check.

Part 3 — Two code bugs

☒ `break_points_won` is always NULL, for every row, silently

In `_side_match_row`, the function computes the opponent's column prefix on its first line:

```
opponent_prefix = "l" if side == "w" else "w"
```

and then never uses it. The break-points calculation instead uses the `opponent` parameter:

```
"break_points_won": (  
    None  
    if _nullable_int(row.get(f"{opponent}_bpFaced")) is None  
    ...  
)
```

But `opponent` is passed a **player ID**, not a prefix — `transform_sackmann_row` calls `_side_match_row(..., opponent=loser_id, ...)`. So the lookup key becomes `"104925_bpFaced"`, a column that does not exist, `.get()` returns `None`, and the whole expression short-circuits to `None`.

Confirmed on a real match. I fed it Sinner d. Djokovic, Madrid 2024, 6-4 7-5:

```
winner aces: 9 | loser aces: 4      (correct)  
winner df: 2 | loser df: 5        (correct)  
winner bp_faced: 4 | loser bp_faced: 7 (correct)  
  
break_points_won winner: None loser: None  
    expected: winner = l_bpFaced - l_bpSaved = 7-5 = 2  
              loser  = w_bpFaced - w_bpSaved = 4-3 = 1
```

Every other stat maps correctly — the winner/loser expansion itself is sound, which is the part I was most worried about. Only this one derived column is broken.

It is currently harmless because no feature in `tennis.yaml` references `break_points_won`. That is also what makes it dangerous: the column sits in `matches`, 100% NULL, looking like a legitimate field. When you eventually build a break-points prop, you will get a table full of nulls and no error.

Fix: change `f"{opponent}_bpFaced"` and `f"{opponent}_bpSaved"` to use `opponent_prefix`. Then backfill.

❌ Your config claims three active stat targets and builds one

```
stat_targets:
  - aces
  - double_faults
  - service_games
```

But `features.targets` defines exactly one:

```
features.targets defined: ['aces']
DECLARED ACTIVE BUT UNDEFINED -> ['double_faults', 'service_games']
```

`build_features.py` iterates `config["targets"].items()`, so it builds aces features and silently ignores the other two. Nothing in `config/_schema.py` cross-validates the two lists — `stat_targets` is just `list[str]` with `min_length=1`, and `targets` is a free-form dict.

So `tennis.yaml` currently asserts something false about your own pipeline. Either define the two missing targets or move them to `disabled_stat_targets`, and add a validator so this can't drift again. Given that `double_faults` and `service_games` have the same shape as `aces`, the definitions are mostly a copy with three column names changed.

Part 4 — The honest limitation in the leakage audit

This is the most interesting thing in the codebase and it deserves a careful explanation, because it is a limitation the tests structurally cannot detect.

Sackmann-format data does not contain match dates. It contains `tourney_date`, the date the *tournament* started. Every match in a two-week Grand Slam shares one date.

If you use `tourney_date` as your event date, then no match in a tournament can use any earlier match from that same tournament as history — a player's third-round performance is invisible to their quarter-final features. Safe, but it throws away a lot of signal.

The agent's solution is `estimate_event_date`, which adds a per-round offset:

```
ROUND_OFFSETS = {"Q1":-1, "Q2":-1, "Q3":-1, "Q4":-1,
                  "R128":0, "R64":1, "R32":2, "R16":3, "QF":4, "SF":5, "F":6}
```

It is documented honestly — the docstring says these "are a transparent approximation based on the round ordering; they are not presented as exact observed dates" — and the config gates it behind `date_tie_policy: prior_event_dates_only`, with `load_feature_config` refusing to run under any other policy.

Why this is the right call. Within a tournament, round order is a genuine chronology: R64 really did happen before R32. Feeding earlier rounds into later-round features is correct and recovers real signal.

Where it gets slippery. The offsets assume one round per day anchored at R128, which is true for nothing. A 250-level event has a 32-player draw starting Monday, so its R32 is day 0 in reality and day 2 by estimate. Grand Slam first rounds span three days. So absolute dates are wrong by up to about three days, in a direction that varies by tournament size.

Two consequences:

`rest_days` is **noisy**. It is computed as days between consecutive estimated dates, so it inherits the error from both endpoints and can be off by several days in either direction. For a feature meant to capture fatigue, that is meaningful noise. It is not leakage and it will not produce fake accuracy — it will just make the feature weaker than it should be. Worth knowing before you conclude that rest doesn't predict serving.

The audit cannot detect date-ordering errors. This is the subtle part. The assertion is `item.event_date < event_date` — comparing *estimated* dates. If two matches are ordered wrongly by the estimate, the assertion still passes, because it is validating internal consistency against the same estimate that produced the ordering. A test that checks your data against your own assumption can only confirm the assumption is applied consistently, never that it is true.

How much risk is that actually? Low, and I want to be precise rather than alarming. For a leak to occur, a player would need two matches where the estimated order reverses the real order — which requires overlapping events, since within one tournament round order is reliable. A singles player is in one tournament at a time. The realistic exposure is Davis Cup and Olympic weeks overlapping tour events, and qualifying-to-main-draw transitions where all four Q-rounds share the -1 offset.

The cheap way to close it entirely. Instead of trusting estimated dates for ordering, order history by `(tourney_date, round_ordinal)` and require that history come from a strictly earlier `tourney_date`, or the same tournament with a strictly earlier round ordinal. That removes the possibility of cross-tournament misordering while keeping the within-tournament signal, and it does not depend on the offsets being numerically right. The offsets then serve only as a display date.

I would not block Checkpoint 7 on this. I would write it down as a known limitation, and revisit it if backtest results ever look better than they should.

Related: the correlation guard is loose

```
def assert_no_target_correlation(..., threshold: float = 0.99):
```

A correlation of 0.99 between a feature and its target means the feature essentially *is* the target. That catches catastrophic leakage and nothing subtler. A rolling ace average should correlate with next-match aces somewhere around 0.4 to 0.6; a value of 0.9 would be a screaming red flag and would sail through this check.

`print_report` does print the observed maximum, which is the right instinct. Read that number after every build. Once you have a real number from real data, tighten the threshold to sit just

above it — an assertion calibrated to your actual data is worth ten times one set at a round number.

Part 5 — Smaller items

- **games_won and sets_won are hardcoded None in the ETL.** They are in `disabled_stat_targets`, so this is consistent — but you *do* store the score string, and both are derivable from it ("6-4 7-5" → 13 games, 2 sets). Two props you can enable for free whenever you want them, and games-won is one of the more liquid tennis markets.
- **Super-tiebreak scores are read as incomplete.** `is_match_completed` counts set scores matching `^(\d+)-(\d+)` and requires a winner to reach `best_of // 2 + 1`. A deciding match tiebreak written as [10-8] doesn't match, so such matches fall one set short and get flagged incomplete, excluding them from history. Rare in ATP singles, common in doubles and some team competitions. Note it before you extend to doubles.
- **Performance is fine for tennis, not for what comes next.** `_strict_prior_aggregate` is a pure-Python loop and `opponent_rolling` runs three full `iterrows()` passes. I measured it: 32,000 rows takes 8 seconds per `rolling_stat` call and 11 per `opponent_rolling`, scaling linearly. Your 2015–2025 ATP history is roughly 64,000 player-rows and the builder makes about 16 transform passes per target, so expect four to six minutes per stat target. Entirely acceptable. When you reach MLB — where a season is far more player-games than a decade of tennis — this becomes the bottleneck, and `opponent_rolling's iterrows()` is the first thing to vectorize.
- **No odds ingestion module.** There is no `etl/odds/` anywhere. Migration 008 built the schema for pre-match odds capture and nothing fills it. If Checkpoint 5 was meant to include live odds snapshots, that work has not happened — and every night without it is a night of tennis prop odds you can never buy back.
- `.cache/` **should be in** `.gitignore`. `DEFAULT_CACHE_DIR = Path(".cache") / "sackmann_kadantte"` will accumulate tens of megabytes of CSVs. Check whether it's ignored before your next commit.

Part 6 — What to do, in order

Step 1 — Settle the data source. Nothing else matters until this is resolved. Find the live successor to the original repository, verify it has data through last week, and repoint the two constants. Add the freshness assertion described in Part 2.

Step 2 — Two fixes. Prompt for the agent:

Two corrections. No new migrations, no schema changes.

1. In `etl/tennis/sackmann.py`, `_side_match_row` computes `opponent_prefix` on its first line and never uses it. The `break_points_won` expression instead interpolates the opponent parameter, which holds a player ID, not a column prefix — so it reads a nonexistent column and `break_points_won` is NULL for every row ever ingested. Change both `row.get(f"{opponent}_bpFaced")` and `row.get(f"{opponent}_bpSaved")` to use `opponent_prefix`. Add a unit test asserting that for a match where the winner faced 4 break points and saved 3, and the loser faced 7 and saved 5, the winner's `break_points_won` is 2 and the loser's is 1.
2. `config/tennis.yaml` lists aces, double_faults, and service_games under `stat_targets`, but `features.targets` defines only aces, so the other two are silently never built. Add definitions for `double_faults` and `service_games` mirroring the aces definition with appropriate source columns and output names, including entries in the descriptions block. Then add a validator to `config/_schema.py` asserting that every name in `stat_targets` has a corresponding key in `features.targets`, and that no name appears in both `stat_targets` and `disabled_stat_targets`. Add a test for each failure mode.

Report which items you completed and list any you did not do and why.

Step 3 — Re-ingest and read the report. After the fixes, re-run ingestion and the feature build, and actually read `print_report`'s output. Specifically:

- Null rates per feature. `rolling_mean_20` and `career_to_date_mean` should be highly null early in a player's career and rarely null later. `head_to_head_matches` should be zero or null for most rows — most pairs have never met.
- The target distribution. Aces per match should center somewhere around 4–7 with a long right tail. A mean near zero means the 18% of rows without serve stats are being mishandled; a mean above 12 means you are looking at a Slam-only subset.
- The maximum absolute target correlation. Write the number down. If it exceeds about 0.7, stop and investigate before training anything.
- The example row. Read it line by line against the descriptions. This is the single best sanity check you have, and it costs a minute.

Step 4 — Then Checkpoint 7, the baseline model. Your project instructions are specific: a rolling average adjusted for context, built first as the benchmark every later model must beat. It should be a short script, and it should read only from `player_game_features` — never from `matches`.

The number to hold in your head: **mean absolute error of 1.5 to 2.5 aces is a realistic, honest baseline.** If your first baseline reports an MAE below 0.5, you have a leak, not a breakthrough. That expectation is the most valuable thing you can carry into the next checkpoint.

Part 7 — Where the agent stands

Pass	Landed	New problems	Honest report
v2	~6 of 23	3	No — 12 silent skips
v3	~19 of 23	1	Yes
v4	17.5 of 18	0	Yes
v5 (Checkpoints 4–6)	Full feature pipeline, leakage-safe	2 bugs, 1 config drift	Not yet assessed

Something changed in this pass. Nobody prompted it to write a perturbation test, to route every primitive through a single audited function, to exclude same-date ties in both directions, or to add a `descriptions` block explaining each feature in plain language. It got the hardest conceptual part of the tennis build right, and it wrote the tests that prove it.

The two bugs are worth noticing for their shape. Both are the same failure it has shown from the beginning: an unused variable sitting next to a wrong one, and a config that declares more than it delivers. It writes correct algorithms and loses track of details in the plumbing around them. That is a much better failure mode than the reverse, and it is why hand-verifying outputs — the four checks in Part 1, and reading `print_report` — remains worth your time even when the code is good.

The thing it could not have caught is the data problem. No test suite tells you your source repository was deleted and your mirror stopped updating in May. That is the kind of judgment that stays yours, and it is the reason to check your inputs before you trust your outputs.